Proyecto Algoritmos y Estructuras de Datos

Lineamiento: *Línea 1, Gestión de Recursos Comunitarios Compartidos*

Integrantes: *Maria Fernanda Morales, Esteban Velasquez, Edward Castillo, Santiago Lopez*

Fase 1 (Analisis)

1. Problema (Necesidad)

1.1. Contexto del lineamiento

Gestión de Recursos Comunitarios Compartidos, el problema, en bibliotecas, salas de estudio, centros culturales y bancos de herramientas gestionan préstamos manualmente (cuadernos, hojas sueltas, acuerdos verbales), lo que genera reservas duplicadas, pérdida de trazabilidad, asignación injusta de turnos y ausencia de datos para decisiones de inversión, se necesita un sistema que administre inventario y usuarios, gestione préstamos/devoluciones con trazabilidad completa, ordene la demanda de forma justa y verificable, y genere información para sustentar decisiones.

1.2. Contexto del cliente

El cliente es la administración de una red de bibliotecas y salas de estudio universitarias, con varias sedes, cada una a cargo de un bibliotecario. Administran tres tipos de recursos, colección bibliográfica, espacios de estudio y equipos tecnológicos. Su labor diaria incluye prestarlos, recibirlos, asignar turnos cuando hay demanda y darles mantenimiento o reposición.

Actualmente todo se maneja de forma manual y descentralizada (cuadernos, hojas de cálculo por sede, acuerdos verbales), sin visión unificada de la red. Esto genera problemas recurrentes: reservas dobles de un mismo espacio, equipos que se pierden sin rastro de quién los tiene, asignación de turnos poco transparente (no por orden de llegada), y falta de datos confiables para decidir compras, reparaciones o bajas al final del semestre.

Como consecuencia, hay pérdida de recursos, percepción de favoritismo, tiempo desperdiciado localizando objetos y decisiones de inversión mal informadas.

1.3. Contexto de la necesidad

La necesidad es tener un sistema centralizado, ordenado y verificable para gestionar recursos, usuarios y préstamos en toda la red. Surge porque la operación ya superó lo que el registro manual puede sostener, afectando sobre todo al bibliotecario (sin margen para corregir errores) y al administrador (que decide compras sin datos confiables).

El ciclo de préstamo es hoy ineficiente porque no se valida disponibilidad contra un registro común, no hay trazabilidad de quién tiene cada recurso, la fila de espera es informal, no se pueden deshacer errores de registro fácilmente, y localizar un recurso requiere revisar listados completos. La información clave a administrar incluye: inventario y estado de recursos, datos de usuarios, préstamos y devoluciones con fechas, colas de espera, categorías del catálogo, sedes de la red e indicadores de uso.

No resolver esto mantiene la pérdida de recursos, la falta de equidad en los turnos y las decisiones de inversión sin sustento. La oportunidad es transformar ese registro disperso en información estructurada que soporte tanto la operación diaria como la toma de decisiones basada en evidencia.

1.4.Pregunta problema

¿Cómo se puede gestionar de forma centralizada, trazable y justa el préstamo de recursos en una red de bibliotecas y salas de estudio de un campus, evitando reservas duplicadas, sabiendo en todo momento quién tiene cada recurso y asignando los turnos por estricto orden de llegada, y a la vez producir información confiable que permita decidir qué adquirir, reparar o dar de baja?

2. Objetivos

2.1. Objetivo general

Diseñar e implementar, en C++ y en Python, un sistema autónomo de gestión de recursos comunitarios para una red de bibliotecas y salas de estudio de campus, que administre el inventario de recursos y usuarios, gestione préstamos y devoluciones con trazabilidad completa, ordene la demanda de forma justa y verificable, y genere información confiable para sustentar decisiones de inversión, utilizando estructuras de datos propias.

2.2. Objetivos específicos

- Modelar e implementar el registro, consulta, actualización y baja de usuarios y recursos aplicando POO con herencia (recurso base → libro, equipo, sala). Su funcionalidad relacionada es el CRUD de usuarios y recursos, además del inventario inicial. Las estructuras de datos que utiliza son matriz, vector dinámico propio y POO+herencia. Corresponde a los módulos M01 y M02.
- Implementar el registro de préstamos y devoluciones conservando el historial completo y navegable en ambos sentidos de cada recurso y usuario. Su funcionalidad relacionada son los préstamos, devoluciones e historial. Las estructuras de datos que utiliza son lista enlazada simple y lista doble/circular. Corresponde al módulo M03.
- Gestionar una lista de espera por recurso que respete estrictamente el orden de llegada. Su funcionalidad relacionada son los turnos justos. La estructura de datos que utiliza es la cola. Corresponde al módulo M04.

- Implementar la reversión de la última operación registrada sin editar datos a mano ni alterar el historial. Su funcionalidad relacionada es la función de deshacer. La estructura de datos que utiliza es la pila. Corresponde al módulo M03.
- Localizar cualquier usuario o recurso por su código en tiempo constante promedio. Su funcionalidad relacionada es la búsqueda inmediata por código. La estructura de datos que utiliza es una tabla hash propia. Corresponde al módulo M05.
- Construir listados de recursos ordenados por distintos criterios (más prestados, más antiguos, categoría, estado) y consultarlos eficientemente. Su funcionalidad relacionada son los listados y consultas. Las estructuras de datos que utiliza son ordenamientos (1 básico + 2 avanzados) y búsqueda binaria. Corresponde al módulo M06.
- Organizar los recursos en una jerarquía de categorías/subcategorías de profundidad arbitraria y consultar todo lo que cuelga de una categoría, con un índice ordenado y balanceado. Su funcionalidad relacionada es la jerarquía e índice del catálogo. Las estructuras de datos que utiliza son árbol binario, BST y AVL. Corresponde al módulo M07.
- Modelar la red de sedes como grafo para determinar, ante un recurso agotado, en qué sede cercana está disponible y por qué camino más corto, además de detectar sedes aisladas. Su funcionalidad relacionada es la disponibilidad entre sedes. Las estructuras de datos que utiliza son grafo (lista de adyacencia) y los algoritmos DFS/BFS. Corresponde al módulo M08.
- Generar informes de uso con indicadores (más prestados, tasa de devolución, demanda por categoría, stock) que sustenten decisiones de inversión. Su funcionalidad relacionada son los reportes, que integran varias estructuras. Corresponde al módulo M09.
- Garantizar que toda la funcionalidad exista y produzca resultados equivalentes en C++ y Python, con pruebas de caso normal, límite y error. Es un requisito transversal que no depende de una estructura de datos específica y aplica a todos los módulos.

3. Alcance

3.1.Descripción de la solución

Vamos a desarrollar un sistema de gestión de préstamos de consola, autónomo y sin conexión, que funciona como el "cerebro" del mostrador de cada biblioteca de la red y, al mismo tiempo, da una visión unificada de todas las sedes. No es una aplicación con interfaz gráfica: es un programa que el bibliotecario opera desde un menú de texto, y que carga y guarda su información en archivos planos (no en un motor de base de datos), manteniéndola en memoria dentro de estructuras de datos construidas por el equipo.

3.2.Recursos

3.2.1. ***Recursos Tecnológicos***

- *Equipos:* Un computador por integrante (4 equipos en total), propiedad de cada integrante aunque también seran facilitados por la universidad, para desarrollo individual y pruebas.
- *Lenguaje de programación*: Python, acorde al curso de Algoritmos y Estructuras de Datos y C++(g++), obliga a manejar memoria y punteros explciitamente
- *IDE / Editor de código*: Visual Studio Code, vinculado al controlador de versiones
- *Control de versiones*: GitHub, para el trabajo colaborativo y el control de cambios del proyecto entre los integrantes.
- *Herramienta de gestión Kanban*: Notion, para implementar la metodología Kanban definida y mantener un flujo de trabajo estable (Backlog, Por hacer, En progreso, En revisión, Hecho).
- *Herramienta de diagramación UML*: Mermaid + StarUML, crea UML y puede hacer diagramas de clases
- *Datos de prueba*: un conjunto de datos pequeño para la Fase 1 de la codificación y pruebas beta de funcionamiento, y posteriormente un conjunto de datos realista de mayor volumen para las siguientes fases del desarrollo, con el fin de medir la eficiencia de búsqueda.
- *Documentación técnica*: se llevará a cabo tanto en Notion, como en el README del repositorio del proyecto en GitHub, documentando cada estructura y función a medida que se desarrolla para facilitar la comprensión de las partes del código desarrolladas individualmente.

3.2.2. ***Recursos humanos***

- *Equipo de desarrollo*: Maria Fernanda Morales, Esteban Velasquez, Edward Castillo y Santiago Lopez, encargados del diseño del modelo de datos, la implementación de los módulos, las pruebas y la documentación.
- *Docente*: Tatiana Cabrera, quien define los entregables por fase, evalúa los avances y guía metodológicamente al equipo.
- *Roles internos*: Distribuir el trabajo de manera eficiente y justa entre los 4 integrantes del equipo apropiándose cada uno de al menos un rol de trabajo donde se destaque su facilidad en este campo y así mismo poder otorgar un plazo de tareas y optimizar el tiempo de entrega.
- *Asignación de roles:* Se aclara que son roles rotativos entrega a entrega para tener un manejo justo de tiempo y trabajo para todo el equipo de desarrollo.

3.3. Tiempo

- En la fase de análisis se trabajará el problema, el alcance, los objetivos, la solución, los recursos y el Kanban. Los responsables son Edward y todo el equipo. Tiene una duración estimada de 3 días y no depende de ninguna fase previa.
- En la fase de planeación se desarrollarán los RQF, las HU, los criterios, los escenarios, la matriz y el diagrama de solución. Los responsables son Edward y María F. Tiene una duración estimada de 3 días y depende de la fase 1.
- En la fase de diseño se definirán los módulos, las estructuras justificadas, el diagrama de clases y el modelo de datos. Los responsables son María F. y todo el equipo. Tiene una duración estimada de 3 días y depende de la fase 2.
- En la fase de desarrollo se construirá el modelo de clases junto con las estructuras lineales en C++ y Python. Los responsables son Esteban (C++), Santiago (Python) y María F. Tiene una duración estimada de 5 días y depende de la fase 3.
- En la fase de pruebas se ejecutarán los casos normales, límite y de error, además de la equivalencia entre C++ y Python y la revisión de memoria. Los responsables son Santiago y Esteban. Tiene una duración estimada de 2 días y depende de la fase 4.
- En la fase de documentación se elaborará la comparativa entre C++ y Python, el README, el informe y la bitácora. Los responsables son Edward y Santiago. Tiene una duración estimada de 2 días y depende de las fases 4 y 5.
- En la fase de entrega y sustentación se subirá el proyecto a e-aulas y se preparará la exposición de 10 minutos. Los responsables son todos los integrantes del equipo. Tiene una duración estimada de 1 día y depende de las fases 1 a 6.

3.4.Módulos

- *El módulo M01 (Gestión de usuarios),* tiene como propósito administrar el padrón de usuarios. Sus funciones principales son registrar, consultar, actualizar, dar de baja y buscar por código. Su usuario es Operario/Admin.
- *El módulo M02 (Recursos e inventario),* tiene como propósito administrar los recursos y su disponibilidad por sede. Sus funciones principales son el CRUD de recursos, la consulta de estado/stock y la matriz de disponibilidad sede×día. Su usuario es Operario/Admin.
- *El módulo M03 (Préstamos, devoluciones y deshacer),* tiene como propósito ejecutar el ciclo del préstamo con historial y función de deshacer. Sus funciones principales son registrar préstamo/devolución, ver historial y deshacer la última operación. Su usuario es Operario.
- *El módulo M04 (Turnos / lista de espera),* tiene como propósito asignar turnos justos por orden de llegada. Sus funciones principales

son encolar solicitud, asignar el siguiente turno y ver la cola. Su usuario es Operario.

- *El módulo M05 (Búsqueda por código),* tiene como propósito localizar en tiempo constante promedio. Sus funciones principales son buscar usuario o recurso por código mediante tabla hash. Su usuario es Operario/Admin.
- *El módulo M06 (Ordenamientos y listados),* tiene como propósito producir vistas ordenadas del inventario. Sus funciones principales son listar por más prestado, antigüedad, categoría o estado, además de búsqueda binaria. Su usuario es Admin.
- *El módulo M07 (Categorías del catálogo),* tiene como propósito organizar el catálogo en jerarquía y mantener un índice ordenado. Sus funciones principales son crear categoría/subcategoría, consultar subárbol y mantener el índice mediante BST/AVL. Su usuario es Operario/Admin.
- *El módulo M08 (Red de sedes),* tiene como propósito responder disponibilidad entre sedes y rutas. Sus funciones principales son buscar la sede más cercana con el recurso, calcular el camino más corto y detectar sedes aisladas mediante DFS/BFS. Su usuario es Operario/Admin.
- *El módulo M09 (Informes de uso),* tiene como propósito sustentar decisiones de inversión con datos. Sus funciones principales son generar indicadores como los más prestados, la tasa de devolución, la demanda por categoría y el stock. Su usuario es Admin.

3.5.Precio

3.5.1. Precio hipotético en el ambiente estudiantil

- *Software*: costo de $0, ya que todas las herramientas que se propones (Git, VS Code, Notion, PseInt) cuentan con versiones gratuitas y educativas suficientes para desarrollar en su totalidad el proyecto.
- *Infraestructura de cómputo*: costo de $0, pues se usan los equipos propios de los integrantes y no se requiere servidor ni nube, únicamente el controlador de versiones pues el sistema es autónomo y funciona con bases de datos locales.
- *Imprevistos*: costo mínimo o nulo, pues es un proyecto universitario sin una producción y distribución del producto a desarollar, por lo que el riesgo financiero es prácticamente nulo.
- Total: Tiene un costo nulo

3.5.2. Precio real en el ambiente profesional

- *Mano de obra*: costo aproximado de 13 a 22 millones, 360h * 35000/h (Salario Desarrollador Junior), 60000/h (Salario Desarrollador Semi-senior)

- *Infraestructura de cómputo*: costo de $0, aplicaciones locales, no es necesario el uso de la nube
- *Herramientas y licencias*: costo aproximado de máximo 500000, aunque la mayoría son g
- *Total*: Un costo de más o menos 13.5 a 22.5 millones de pesos, dependiendo de el nivel de los desarrolladores

4. **Metodología**

El equipo trabajara en dos diferentes etapas, primero el diseño conjunto y luego la implementación por entregas.

En la etapa de diseño, que se hará antes de escribir la primera línea de código, el equipo se reunirá para acordar el modelo de datos que se usara, que información describe a un usuario, un recurso, una sede, y como se relacionan entre sí. Este acuerdo se documenta mediante un diagrama de flujo o pseudocodigo central y compartido, para que cada integrante trabaje sobre él y el sistema funcione de manera coherente.

En la parte de implementación, el trabajo se organiza mediante unos entregables interpuestos por la líder del curso.

- En la primera fase se construye la funcionalidad esencial sobre un conjunto de datos pequeños, para validar de esta manera que el diseño funciona correctamente antes de invertir tiempo y esfuerzo con datos de mayor volumen y poder corregir errores básicos del sistema.
- A partir de la entrega 2, se incorpora el conjunto de datos realistas exigido para el proyecto, momento en el que el equipo mide el desempeño del sistema, específicamente que la búsqueda por código funcione con fluides y sea eficiente.

Cada estructura y función se documentará en el momento que se escribe, es importante exponer que hace, que recibe, que devuelve y por qué se elegio dicha estructura, para que de esta manera el código sea legible por cualquier persona externa del proyecto. Esto incluye, comentarios dentro del código como también una breve guía que explique cómo está organizado el sistema, de modo que revisar y dar mantenimiento no dependa exclusivamente de los creadores del proyecto.

Dale click para ver nuestro tablero Kanban Board proyecto

Fase 2 (Planeación)

1. **Requerimientos funcionales (RQF)**

1.1.Módulo M01 Gestión de usuarios

- *RQF-001* Registrar usuario. Descripción: permite dar de alta un nuevo usuario en el sistema capturando id_usuario, nombre y correo. Usuario: Operario/Admin.

- *RQF-002* Consultar usuario por código. Descripción: permite ubicar un usuario existente ingresando su id_usuario y visualizar sus datos completos. Usuario: Operario/Admin.
- *RQF-003* Actualizar datos de usuario. Descripción: permite modificar el nombre o correo de un usuario ya registrado. Usuario: Operario/Admin.
- *RQF-004* Dar de baja usuario. Descripción: permite eliminar o inactivar un usuario del padrón, validando que no tenga préstamos activos pendientes. Usuario: Admin.

1.2.Módulo M02 Recursos e inventario

- *RQF-005* Registrar recurso. Descripción: permite dar de alta un nuevo recurso (libro, equipo o sala) aplicando herencia desde la clase base Recurso, con título, categoría y subcategoría. Usuario: Operario/Admin.
- *RQF-006* Consultar recurso por código. Descripción: permite ubicar un recurso mediante su id_recurso y visualizar su información y disponibilidad. Usuario: Operario/Admin.
- *RQF-007* Actualizar recurso. Descripción: permite modificar los datos de un recurso existente (título, categoría, subcategoría). Usuario: Operario/Admin.
- *RQF-008* Dar de baja recurso. Descripción: permite retirar un recurso del inventario cuando está dañado o se decide reponerlo. Usuario: Admin.
- *RQF-009* Consultar disponibilidad por sede. Descripción: permite verificar la cantidad_total y cantidad_disponible de un recurso en una sede específica. Usuario: Operario/Admin.
- *RQF-010* Generar matriz de disponibilidad sede×día. Descripción: permite construir y consultar una matriz que cruza sedes contra días para visualizar la disponibilidad proyectada. Usuario: Admin.

1.3.Módulo M03 Préstamos, devoluciones y deshacer

- *RQF-011* Registrar préstamo. Descripción: permite crear un nuevo préstamo validando disponibilidad en inventario y asociando id_usuario, id_recurso, id_sede y fecha_prestamo. Usuario: Operario.
- *RQF-012* Registrar devolución. Descripción: permite cerrar un préstamo activo registrando la fecha_devolucion_real y actualizando el estado a devuelto. Usuario: Operario.
- *RQF-013* Consultar historial de préstamos de un usuario. Descripción: permite recorrer en ambos sentidos (lista doble) el historial completo de préstamos asociados a un usuario. Usuario: Operario/Admin.
- *RQF-014* Consultar historial de préstamos de un recurso. Descripción: permite ver todos los préstamos históricos de un recurso específico. Usuario: Operario/Admin.
- *RQF-015* Deshacer última operación. Descripción: permite revertir el último préstamo o devolución registrado usando una pila, sin editar datos manualmente ni alterar el resto del historial. Usuario: Operario.

1.4.Módulo M04 Turnos / lista de espera

- *RQF-016* Registrar solicitud en cola de espera. Descripción: permite encolar una solicitud (id_solicitud, id_usuario, id_recurso, fecha_solicitud) cuando el recurso no está disponible. Usuario: Operario.
- *RQF-017* Asignar siguiente turno. Descripción: permite desencolar y asignar el recurso a la siguiente solicitud en estricto orden de llegada, apenas quede disponible. Usuario: Operario.
- *RQF-018* Consultar cola de espera de un recurso. Descripción: permite visualizar el orden completo de solicitudes pendientes para un recurso determinado. Usuario: Operario.

1.5.Módulo M05 Búsqueda por código

- *RQF-019* Buscar usuario por código. Descripción: permite localizar un usuario en tiempo constante promedio mediante una tabla hash propia. Usuario: Operario/Admin.
- *RQF-020* Buscar recurso por código. Descripción: permite localizar un recurso en tiempo constante promedio mediante una tabla hash propia. Usuario: Operario/Admin.

1.6.Módulo M06 Ordenamientos y listados

- *RQF-021* Listar recursos más prestados. Descripción: permite ordenar y visualizar los recursos según su cantidad de préstamos históricos. Usuario: Admin.
- *RQF-022* Listar recursos por antigüedad. Descripción: permite ordenar los recursos según su fecha de ingreso al inventario. Usuario: Admin.
- *RQF-023* Listar recursos por categoría o estado. Descripción: permite generar listados ordenados filtrando por categoría o por estado (disponible/prestado/de baja). Usuario: Admin.
- *RQF-024* Buscar recurso en listado ordenado. Descripción: permite aplicar búsqueda binaria sobre un listado ya ordenado para ubicar un recurso específico. Usuario: Admin.

1.7.Módulo M07 Categorías del catálogo

- *RQF-025* Crear categoría/subcategoría. Descripción: permite dar de alta una categoría o subcategoría dentro de la jerarquía del catálogo, con profundidad arbitraria. Usuario: Operario/Admin.
- *RQF-026* Consultar subárbol de una categoría. Descripción: permite visualizar todas las subcategorías y recursos que cuelgan de una categoría dada. Usuario: Operario/Admin.
- *RQF-027* Mantener índice ordenado de categorías. Descripción: permite insertar, buscar y balancear el índice de categorías mediante un BST/AVL para mantener consultas eficientes. Usuario: Admin.

1.8.Módulo M08 Red de sedes

- *RQF-028* Consultar sede más cercana con recurso disponible. Descripción: permite, ante un recurso agotado en la sede actual,

identificar la sede más cercana donde sí esté disponible. Usuario: Operario/Admin.

- *RQF-029* Calcular camino más corto entre sedes. Descripción: permite determinar la ruta más corta entre la sede actual y la sede con el recurso disponible, usando BFS/DFS sobre el grafo de sedes. Usuario: Operario/Admin.
- *RQF-030* Detectar sedes aisladas. Descripción: permite identificar sedes que no tienen conexión con el resto de la red mediante recorrido del grafo. Usuario: Admin.

1.9.Módulo M09 Informes de uso

- *RQF-031* Generar informe de recursos más prestados. Descripción: permite producir un reporte con los recursos de mayor demanda histórica. Usuario: Admin.
- *RQF-032* Generar informe de tasa de devolución. Descripción: permite calcular y mostrar el porcentaje de préstamos devueltos a tiempo frente al total. Usuario: Admin.
- *RQF-033* Generar informe de demanda por categoría. Descripción: permite mostrar qué categorías del catálogo concentran más préstamos. Usuario: Admin.
- *RQF-034* Generar informe de stock disponible. Descripción: permite consultar el estado general de cantidad_disponible frente a cantidad_total por recurso y sede. Usuario: Admin.

2. Historia de usuario (HU)

2.1.*HU-001 (RQF-001):* Como operario, quiero registrar un nuevo usuario con su nombre y correo, para poder habilitarlo a solicitar préstamos.

2.2.*HU-002 (RQF-002):* Como operario, quiero consultar los datos de un usuario por su código, para verificar su identidad antes de prestarle un recurso.

2.3.*HU-003 (RQF-003):* Como operario, quiero actualizar los datos de un usuario, para mantener su información de contacto al día.

2.4.*HU-004 (RQF-004):* Como administrador, quiero dar de baja a un usuario sin préstamos activos, para depurar el padrón de usuarios inactivos.

2.5.*HU-005 (RQF-005):* Como operario, quiero registrar un nuevo recurso indicando su tipo (libro, equipo o sala), para incorporarlo al inventario de la sede.

2.6.*HU-006 (RQF-006):* Como operario, quiero consultar un recurso por su código, para conocer su disponibilidad antes de prestarlo.

2.7.*HU-007 (RQF-007):* Como operario, quiero actualizar la información de un recurso, para corregir datos incorrectos del catálogo.

2.8.*HU-008 (RQF-008):* Como administrador, quiero dar de baja un recurso deteriorado, para que deje de ofrecerse en préstamo.

2.9.*HU-009 (RQF-009):* Como operario, quiero consultar cuántas unidades de un recurso hay disponibles en mi sede, para decidir si puedo prestarlo de inmediato.

2.10. *HU-010 (RQF-010):* Como administrador, quiero ver una matriz de disponibilidad por sede y día, para planear reposiciones con anticipación.

2.11. *HU-011 (RQF-011):* Como operario, quiero registrar un préstamo validando la disponibilidad real del recurso, para evitar reservas duplicadas.

2.12. *HU-012 (RQF-012):* Como operario, quiero registrar la devolución de un recurso, para liberar el inventario y actualizar el historial.

2.13. *HU-013 (RQF-013):* Como operario, quiero ver el historial completo de préstamos de un usuario, para resolver dudas sobre su actividad.

2.14. *HU-014 (RQF-014):* Como operario, quiero ver el historial completo de préstamos de un recurso, para saber quiénes lo han solicitado.

2.15. *HU-015 (RQF-015):* Como operario, quiero deshacer la última operación registrada por error, para corregirla sin manipular datos a mano.

2.16. *HU-016 (RQF-016):* Como operario, quiero encolar la solicitud de un usuario cuando el recurso no está disponible, para respetar su turno.

2.17. *HU-017 (RQF-017):* Como operario, quiero que el sistema asigne automáticamente el siguiente turno en la cola, para eliminar la percepción de favoritismo.

2.18. *HU-018 (RQF-018):* Como operario, quiero ver la cola de espera de un recurso, para informar a los usuarios cuántas personas hay antes que ellos.

2.19. *HU-019 (RQF-019):* Como operario, quiero buscar un usuario por su código de forma instantánea, para agilizar la atención en el mostrador.

2.20. *HU-020 (RQF-020):* Como operario, quiero buscar un recurso por su código de forma instantánea, para no revisar listados completos.

2.21. *HU-021 (RQF-021):* Como administrador, quiero ver un listado de los recursos más prestados, para priorizar su reposición.

2.22. *HU-022 (RQF-022):* Como administrador, quiero ver los recursos ordenados por antigüedad, para identificar cuáles considerar dar de baja.

2.23. *HU-023 (RQF-023):* Como administrador, quiero filtrar y ordenar recursos por categoría o estado, para generar reportes específicos.

2.24. *HU-024 (RQF-024):* Como administrador, quiero buscar un recurso dentro de un listado ya ordenado, para encontrarlo de forma más eficiente.

2.25. *HU-025 (RQF-025):* Como operario, quiero crear categorías y subcategorías del catálogo, para mantener el inventario organizado jerárquicamente.

2.26. *HU-026 (RQF-026):* Como operario, quiero consultar todo lo que cuelga de una categoría, para ver de un vistazo todos los recursos relacionados.

2.27. *HU-027 (RQF-027):* Como administrador, quiero que el índice de categorías se mantenga balanceado, para garantizar consultas rápidas aun con muchas categorías.

2.28. *HU-028 (RQF-028):* Como operario, quiero saber en qué sede cercana está disponible un recurso agotado en la mía, para orientar al usuario.

2.29. *HU-029 (RQF-029):* Como operario, quiero conocer el camino más corto hacia esa sede, para dar una indicación práctica al usuario.

2.30. *HU-030 (RQF-030):* Como administrador, quiero detectar sedes aisladas de la red, para corregir la conectividad logística entre sedes.

2.31. *HU-031 (RQF-031):* Como administrador, quiero un informe de los recursos más prestados, para sustentar decisiones de compra.

2.32. *HU-032 (RQF-032):* Como administrador, quiero conocer la tasa de devolución a tiempo, para evaluar el comportamiento de los usuarios.

2.33. *HU-033 (RQF-033):* Como administrador, quiero ver la demanda por categoría, para orientar el presupuesto de adquisiciones.

2.34. *HU-034 (RQF-034):* Como administrador, quiero ver el stock disponible frente al total, para decidir reposiciones antes de que se agote un recurso.

3. **Diagrama de Flujo (Solución completa), PSeInt**

   https://drive.google.com/file/d/1VUPDZ-lJp4FgfUqLo-65vYzP454TOXZm/view?usp=sharing

   El diagrama de flujo fue realizado por Claude IA, mediante instrucciones claras de como deseamos sea la estructura y que haga nuestro proyecto.